Quick CEBD - Octobre 2024

Durée : 1 Heure / Document autorisé : une feuille A4, recto-verso manuscrite

Position du problème : les compétitions sportives

Le **modèle relationnel** est donné ci-dessous :

LesDisciplines (nomDi)
/ <n> ∈ LesDisciplines ⟺ la discipline a un nom n */*
LesEpreuves (numEp, nomEp, formeEp, categorieEp, nbSportifsEp, dateEp, nomDi)
/ <no, n, f, c, nb, da, di> ∈ LesEpreuves ⟺ no est le numéro d'épreuve du nom n, forme f (individuelle, par equipe ou par couple), catégorie c (féminin, masculin ou mixte), un nombre de sportifs nb et une date d. L'épreuve fait partie de la discipline di */*
LesSportifs (numSp, nomSp, prenomSp, pays, categorieSp, dateNaisSp)
/ <no, n, p, pa, c, d> ∈ LesSportifs ⟺ no est le numéro de sportif, avec un nom n, un prénom p, un pays pa une date de naissaince d et une catégorie c (feminin ou masculin) */*
LesEquipiers (numSp, numEq)
/ <nS, nE> ∈ LesEquipiers ⟺ le sportif de numéro nS fait partie de l'équipe nE */*
LesInscriptions (numIn, numEp)
/ <i, e> ∈ LesInscriptions ⟺ le participant i s'inscrit à l'épreuve e */*
LesResultats (numEp, gold, silver, bronze)
/ <e, o, a, b> ∈ LesResultats ⟺ le participants o, a et b ont remporté les médailles d'or, argent et bronze respectivement à l'épreuve e */*
Les contraintes de domaine :

domain(numEq) = [1000, 1500] domain(numSp) = [1, 100]
domaine (dateNaisSp) = date(dateEp) = Date */* par ex. 24/11/2020 */*
domaine (formeEp) = {'individuelle', 'par equipe', 'par couple'}
domaine (categorieEp) = {'feminin', 'masculin', 'mixte'}; domaine (categorieSp) = {'feminin', 'masculin'}
domaine (nomDi) = domaine (nomEp) = domaine (nomSp) = domaine (prenomSp) = domaine (pays) = chaînes de car.
le reste des attributs sont des entiers positifs

Contraintes d'intégrité référencielle :

LesEpreuves[nomDi] ⊆ LesDisciplines[nomDi], LesEquipiers[numSp] ⊆ LesSportifs[numSp],
LesInscriptions[numEp] = LesEpreuves[numEp], LesInscriptions[numIn] ⊆ LesSportifs[numSp] ∪ LesEquipiers[numEq],
LesResultats[gold, numEp] ⊆ LesInscriptions[numIn, numEp], LesResultats[silver, numEp] ⊆ LesInscriptions[numIn, numEp]
LesResultats[bronze, numEp] ⊆ LesInscriptions[numIn, numEp], LesSportifs[numSp] ∩ LesEquipiers[numEq] = ∅

Autres contraintes d'integrité :

- nomDi not null dans la relation LesEpreuves
- Au moins deux sportifs par équipe et au moins trois inscriptions par épreuve.
- Il n'y a pas d'ex-aequo et toutes les médailles sont attribuées.
- Un participant ne peut gagner qu'une médaille pour une épreuve donnée.
- Si une médaille est attribuée alors, nécessairement, la médaille de rang supérieur est attribuée.
- Un sportif participant à une épreuve individuelle ne peut s'inscrire qu'à celle de sa catégorie (masculin ou féminin).
- Tous les sportifs d'une même équipe représentent le même pays.
- Une épreuve individuelle n'admet que des sportifs.
- Une épreuve par équipe n'admet que des équipes.

1 Contraintes

Question 1 (5 points) :
Comment peut-on contrôler au niveau de l'implémentation de la base de données les contraintes suivantes ? **Fournir le code SQL avec la syntaxe SQlite** permettant de coder les contraintes (pas besoin du CREATE TABLE avec tous les attributs). Donner un nom aux contraintes et specifiez le type de contrainte.

1. domain(numEq) = [1000, 1500] et domain(numSp) = [1, 100]
   Il s'agit de constraintes de **domaine**

   ```
   CONSTRAINT dom_numEq CHECK (numEq>=1000 AND numEq<=1500);
   CONSTRAINT dom_numSp CHECK (numEq>=1 AND numEq<=150);
   ```

2. LesResultats[gold, numEp] ⊆ LesInscriptions[numIn, numEp]
   Il s'agit d'une contrainté d'**intégrité référentielle** ou clé etrangere.

   ```
   CONSTRAINT fk_resultats_gold_numEP
      FOREIGN KEY (gold, numEp) REFERENCES LesInscriptions(numIn, numEp);
   ```

3. Un sportif participant à une épreuve individuelle ne peut s'inscrire qu'à celle de sa catégorie (masculin ou féminin). Utilisez un trigger en remplissant `evenement`, `requete` et `message`.

   ```
   CREATE TRIGGER InscriptionsCatIndiv
           <evenement>
           WHEN EXISTS(<requete>)
           BEGIN
                   SELECT RAISE(ABORT, <message>);
           END;
   ```

   Cette contrainte ne peut pas etre exprimée en langage relationnel. On utilise un Trigger.

   ```
   CREATE TRIGGER InscriptionsCatIndiv
      AFTER INSERT ON LesInscriptions
      WHEN
         EXISTS(
            SELECT *
            FROM LesSportifs S JOIN LesInscriptions I ON (S.numSp = I.numIn)
                               JOIN LesEpreuves E ON (I.numEp = E.numEp AND
                                                      S.categorieSp <> E.categorieEp)
            WHERE E.formeEp='individuelle'
   )
   BEGIN
      SELECT RAISE(ABORT, 'Sportif pas dans sa categorie.');
   END;
      -- Autre version avec BEFORE
   CREATE TRIGGER InscriptionsCatIndiv
      BEFORE INSERT ON LesInscriptions
      WHEN EXISTS(
         SELECT *
         FROM LesEpreuves E JOIN LesSportifs S ON (E.numEp = NEW.numEp AND
                                                   S.numSp = NEW.numIn AND
                                                   S.categorieSp <> E.categorieEp)
         WHERE E.formeEp='individuelle'
         )
         BEGIN
            SELECT RAISE(ABORT, 'Sportif pas dans sa categorie.');
   END;
   ```

2 Expression de requêtes et vues

Question 2 (10 points) :
Exprimer dans l'**algèbre relationnelle**, lorsque c'est possible, et en **SQL** les requêtes ci-dessous.

1. Donner le nom, prénom des sportifs français qui sont inscrits à au moins une épreuve.

   (LesSportifs :pays='France'(numIn=numSp)*LesInscriptions)[nomSp,prenomSp]
   (LesSportifs(numIn=numSp)*LesInscriptions) :pays='France'[nomSp,prenomSp]

   ```
   SELECT DISTINCT nomSp, prenomSp
   FROM LesSportifs JOIN LesInscriptions ON(numSp=numIn)
   WHERE pays='France';
   ```

2. Une vue **LesEquipes** (numEq, nbEquipiersEq). *nbEquipiersEq* correspond au nombre de sportifs d'une équipe.

   Pas possible le COUNT en Algèbre

   ```
   CREATE VIEW IF NOT EXISTS LesEquipes (numEq, nbEquipiersEq) AS
      SELECT numEq, COUNT (numSp) AS nbEquipiersEq
      FROM LesEquipiers
      GROUP BY numEq;
   ```

3. Donner les numéros des equipes qui ont gagné au moins une médaille d'or. Pour chaque équipe, donner le numéro, nom et prenom de chaque membre.

   Algèbre :
   (LesResultats(gold=numEq)*LesEquipiers*LesSportifs)[numEq, numSp, nomSp, prenomSp]

   ```
   -- OK si on utilise la vue LesEquipes et LesSportifs
   SELECT DISTINCT numEq, numSp, nomSp, prenomSp
   FROM LesEquipiers JOIN LesResultats ON (numEq = gold)
                     JOIN LesSportifs USING (numSp);
   ```

4. Donner les pays qui participent (inscription) à toutes les épreuves. **Piste** : un pays peut être représenté dans une épreuve via une équipe ou de manière individuelle via un sportif.

   *On aurait pu aussi utiliser la vues LesEquipes
   Algèbre :
   E ⟵ LesEquipiers
   S ⟵ LesSportifs
   I ⟵ LesInscriptions
   EpreuvesPaysViaSportifs ⟵ (I(numIn=numSp)*S)[numEp, pays]
   EpreuvesPaysViaEquipes ⟵ (I(numIn=numEq)*E*S)[numEp, pays]
   EpreuvesParPays ⟵ EpreuvesPaysViaSportifs ∪ EpreuvesPaysViaEquipes
   PaysToutesEpreuves ⟵ EpreuvesParPays / EpreuvesParPays[numEp]

   ```
   WITH EpreuvesTotal AS (
      SELECT COUNT (numEp) AS nbEpreuvesTotal
      FROM LesEpreuves
   ),
   EpreuvesParPays AS (
      SELECT numEp, pays
      FROM LesInscriptions JOIN LesSportifs ON (numIn = numSp)
      UNION
      SELECT numEp, pays
      FROM LesInscriptions JOIN LesEquipiers ON (numIn=numEq)
                           JOIN LesSportifs USING (numSp)
   ),
   NbEpreuvesParPays AS (
      SELECT pays, COUNT(DISTINCT numEp) AS nbEpreuvesPays
   ```

```
    FROM EpreuvesParPays
    GROUP BY pays
)
SELECT pays
FROM NbEpreuvesParPays JOIN EpreuvesTotal ON (nbEpreuvesTotal = nbEpreuvesPays);
```

3 Compréhension de requêtes

Soient R(A, B) et S(A, C) deux relations dont les valeurs sont données dans les tables suivantes :

R

A	B
2	
9	12
7	12

S

A	C
7	b
2	d
7	f

On a de plus la contrainte d'intégrité référentielle : S[A] $\subseteq$ R[A] (option ON DELETE CASCADE).

Question 3 (5 points) :
Pour chaque expression SQL donnée ci-dessous, indiquer si elle est correcte ou non ; si oui, donner sous forme tabulaire le schéma et la valeur de la relation qu'elle décrit, sinon indiquer l'erreur.

1. SELECT AVG(B) AS avgB
   FROM R ;

avgB
12

2. SELECT A, C
   FROM R LEFT JOIN S USING (A)
   WHERE B IS NOT NULL ;

A	C
9	
7	b
7	f

3. DELETE FROM R WHERE A=7 ;

R

A	B
2	
9	12

S

A	C
2	d

4. SELECT C, MIN(A)
   FROM S
   GROUP BY A ;

   Erreur. C is not a GROUP BY expression